MAXAM | FANEXA

MANUAL TÉCNICO

Frontend

Sistema de Control de Viáticos (SCV)

Gestión de viajes corporativos y rendición de gastos

Autor: Mateo Alejandro Merino Vidal

Empresa: MAXAM FANEXA

Componente: Cliente web (React, Vite y Tailwind CSS)

Versión: 2.1

Fecha: 24 de octubre de 2026

Índice

1. Introducción **3**
1.1. Propósito del documento . . . 3
1.2. Descripción general . . . 3

2. Stack tecnológico **3**

3. Arquitectura **4**
3.1. Capas . . . 4
3.2. Atomic Design . . . 5
3.3. Estructura de carpetas . . . 5

4. Instalación y configuración **6**
4.1. Requisitos . . . 6
4.2. Comandos . . . 6
4.3. Variables de entorno . . . 6
4.4. Imagen de ingreso . . . 6

5. Rutas y control de acceso **6**
5.1. Guardas . . . 6
5.2. Rutas por perfil . . . 7

6. Gestión de sesión **8**
6.1. Token y renovación . . . 8
6.2. Sesión expirada . . . 9
6.3. Cambio de contraseña obligatorio . . . 9

7. Gestión de estado **9**

8. Capa de servicios **10**

9. Hooks de negocio **10**

10. Componentes **11**
10.1. Componentes transversales . . . 11
10.2. Componentes por dominio . . . 12

11. Sistema de diseño **12**
11.1. Paleta de colores . . . 12
11.2. Colores de estado . . . 13
11.3. Ventanas emergentes . . . 13

12. Funcionalidades destacadas **14**
12.1. Planilla de rendición . . . 14
12.2. Categorías de gasto . . . 14
12.3. Motivo del viaje . . . 15
12.4. Monto del viaje . . . 15
12.5. Control de gasto diario . . . 15
12.6. Tipo de cambio . . . 15

12.7. Retenciones de gastos sin factura . . . 15
12.8. Digitalización de comprobantes . . . 15
12.9. Revisión y observaciones . . . 16
12.10. Plazos y reemplazos . . . 16
12.11. Administración . . . 16
12.12. Casquito . . . 17

13. Documentación de componentes **17**

14. Dependencias y seguridad **17**

15. Pruebas automáticas **17**
15.1. Integración continua . . . 18

16. Compilación y despliegue **18**

17. Convenciones **18**

1. Introducción

1.1 Propósito del documento

Este manual describe la construcción interna del cliente web del Sistema de Control de Viáticos (SCV) de MAXAM FANEXA. Está dirigido a desarrolladores que necesiten instalar, mantener o extender la interfaz, y documenta su arquitectura, organización de carpetas, control de acceso, manejo de sesión, componentes, sistema de diseño y despliegue.

1.2 Descripción general

El cliente es una aplicación de una sola página que atiende a seis perfiles: empleado, supervisor, aprobador, tesorero, revisor y administrador. Cada perfil tiene su propio menú y sus propias pantallas, y todas consumen la API REST del backend. La interfaz es adaptable a escritorio y celular.

Perfil	Funciones principales en la interfaz
Empleado	Crea viajes, registra gastos y facturas, finaliza la rendición, pide reemplazos y ampliaciones de plazo, descarga su planilla
Supervisor	Revisión previa de viajes y revisión de rendiciones de su alcance
Aprobador	Aprueba viajes y revisa rendiciones con alcohol
Tesorero	Asigna y aprueba los fondos de cada viaje
Revisor	Revisión final, solicitudes de plazo y de reemplazo
Administrador	Resumen general, usuarios, cargos, secciones, historial de accesos y recordatorios

2. Stack tecnológico

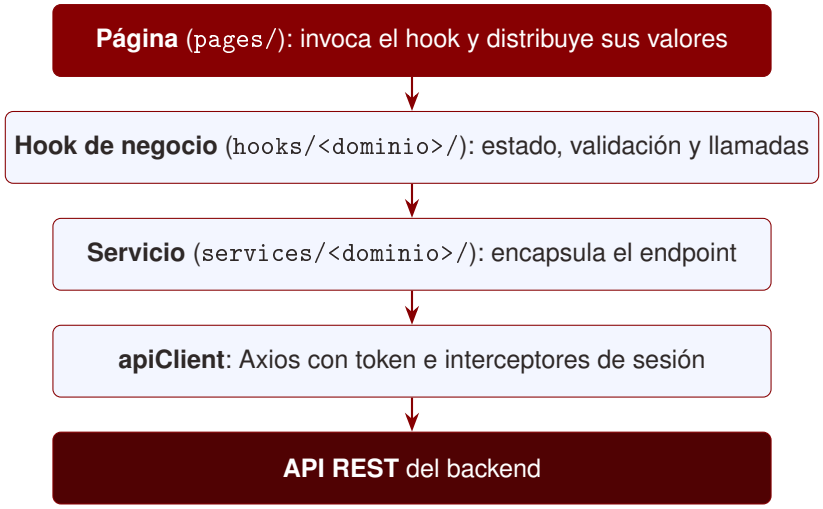
Tecnología	Propósito
React 19	Interfaz mediante componentes funcionales y hooks
Vite	Servidor de desarrollo y compilación
Tailwind CSS 4	Utilidades de maquetado
React Router DOM 7	Enrutamiento y protección por rol
Framer Motion	Animaciones de páginas y ventanas emergentes
Axios	Cliente HTTP con interceptores de sesión
Recharts	Gráficos del panel administrativo
ExcelJS y FileSaver	Planilla de rendición y exportaciones a Excel
Lucide React	Iconografía
jwt-decode	Lectura del rol y del motivo de cambio de contraseña desde el token

Tecnología	Propósito
Storybook	Documentación interactiva de `components/ui`, de los átomos y moléculas de `features` y de los organismos de la carga de facturas
ESLint	Análisis estático del código
Vitest y Testing Library	Pruebas unitarias de hooks, componentes y utilidades
Playwright	Pruebas de extremo a extremo en un navegador real
GitHub Actions	Ejecución automática de las pruebas en cada envío al repositorio

3. Arquitectura

3.1 Capas

El proyecto combina una organización por dominios con una jerarquía de componentes basada en Atomic Design. La lógica de negocio vive en los hooks; los componentes solo presentan.



Capa	Recibe	Entrega	Restricción
Servicio	Parámetros primitivos	Datos o `{error}`	No mantiene estado
Hook	Identificadores de ruta	Estado y manejadores	No renderiza JSX
Página	Parámetros de ruta	Árbol de componentes	No llama servicios
Organismo	Props	Fragmento de interfaz	No conoce hooks de negocio

Ningún componente de presentación importa directamente un servicio. Un servicio típico normaliza la respuesta para que el hook solo pregunte por `error`:

```
export const getTripHistory = async (page, limit, filter) => {
  try {
    const response = await apiClient.get('/trip/history', {params: {pagina: page, limite: limit,
    filtro: filter}});
    return response.data;
  }
  catch (error) {
    return {error: error.response?.data?.error || 'Error al obtener el historial'};
  }
};
```

3.2 Atomic Design

Nivel	Definición	Ejemplos
Átomo	Elemento indivisible sin lógica de negocio	`TripRoute`, `TripStatusBadge`, `ExpenseObsButton`
Molécula	Composición que resuelve una unidad funcional	`ExpenseBudgetBar`, `DateRangeFilter`, `ReturnReviewButton`
Organismo	Bloque autónomo y complejo	`ExpenseInvoicedTable`, `ReviewFilters`, `UserFormModal`

3.3 Estructura de carpetas

```
src/
|-- assets/              Imágenes propias (foto-login.jpg)
|-- components/ui/       Componentes genéricos transversales
|-- features/            Dominios: admin, approval, expense, trip, user
|   `-- <dominio>/
|       |-- atoms/  molecules/  organisms/
|       |-- hooks/       Lógica presentacional
|       `-- constants/   Configuración visual del dominio
|-- hooks/               Hooks de negocio: admin, approval, expense, trip, user, shared
|-- pages/               Páginas asociadas a rutas, y pages/hooks
|-- services/            Acceso a la API por dominio y shared/apiClient.js
|-- layouts/             Navbar, Footer y menús por rol
|-- utils/               Funciones puras (fechas, código de viaje, cantidades, retenciones,
|                        IVA de facturas, compresión de imágenes)
|-- constants/           Paleta, rutas, inicio por rol, monedas, nombres y colores de estado
|-- test/setup.js        Configuración de las pruebas unitarias
|-- App.jsx              Rutas y guardas de acceso
`-- main.jsx             Punto de entrada
e2e/                     Pruebas de extremo a extremo (Playwright)
.github/workflows/       Pruebas automáticas en GitHub Actions
```

Todas las páginas, salvo la de ingreso, se cargan con `React.lazy`: cada ruta genera su propio fragmento de JavaScript y las librerías pesadas (ExcelJS, Recharts) no forman parte del paquete inicial.

4. Instalación y configuración

4.1 Requisitos

- Node.js 22.22 o superior y npm (las herramientas de pruebas lo piden; la aplicación funciona desde 22.12).
- El backend en ejecución y accesible desde el navegador.

4.2 Comandos

```
npm install
npm run dev               # servidor de desarrollo (http://localhost:5173)
npm run build             # compilación para producción (carpeta dist/)
npm run preview           # previsualización de la compilación
npm run lint              # análisis estático
npm run storybook         # documentación de componentes (http://localhost:6006)
npm test                  # pruebas unitarias (Vitest)
npm run test:e2e          # pruebas de extremo a extremo (Playwright)
```

4.3 Variables de entorno

La única variable es `VITE_API_URL`, la dirección del backend. Se define en el archivo `.env` en desarrollo y en las variables del proyecto en Vercel; un cambio requiere volver a compilar.

Etapa	Valor de `VITE_API_URL`
Local	`http://localhost:5000`
Piloto	Dirección del servicio en Render
VPS	Dominio de la API, por ejemplo `https://api.tuempresa.com`

4.4 Imagen de ingreso

La imagen de la pantalla de ingreso es el archivo `src/assets/foto-login.jpg`, que se incluye en la compilación y no depende de internet. Para cambiarla basta con reemplazar el archivo manteniendo el nombre exacto (Linux y Vercel distinguen mayúsculas y extensión). Se carga con `import.meta.glob`, de modo que si el archivo aún no está en el repositorio la compilación no falla y se usa la imagen anterior.

5. Rutas y control de acceso

5.1 Guardas

Componente	Comportamiento
`ProtectedRoute`	Exige sesión y que el rol del token esté entre `allowedRoles`; si no, lleva al inicio del rol del usuario
`LoginRoute`	Con sesión activa, la pantalla de ingreso redirige al inicio del rol
`TreasurerRoute`	Permite las pantallas de tesorería solo a quien tiene el cargo de tesorero
`PasswordChangeGate`	Muestra en cualquier pantalla la ventana de cambio de contraseña obligatorio

Cada rol entra a la primera opción de su menú (`constants/roleHome.js`), y las rutas con parámetros se construyen con funciones de `constants/routes.js` (`tripPath`, `supervisorTripReviewPath`, entre otras) en lugar de textos sueltos.

5.2 Rutas por perfil

Ruta	Página
Empleado (rol 3)	
`/dashboard/empleado`	Panel: borradores, viajes en curso, reemplazos y recientes
`/dashboard/empleado/crear-viaje`	Creación de viaje
`/dashboard/empleado/viaje/:id`	Detalle del viaje según su estado
`/dashboard/empleado/viaje/:id/editar`	Edición de borrador o viaje rechazado
`/dashboard/empleado/viaje/:id/registrar-gasto`	Registro de gastos sin factura
`/dashboard/empleado/viaje/:id/subir-factura`	Carga y lectura de facturas
`/dashboard/empleado/gasto/:id`	Detalle de un gasto
`/dashboard/empleado/historial`	Historial de viajes
Supervisor (rol 2)	
`/dashboard/supervisor`	Rendiciones pendientes
`/dashboard/supervisor/historial`	Historial de rendiciones revisadas
`/dashboard/supervisor/revision/:id`	Revisión de una rendición
`/dashboard/supervisor/viajes-pendientes`	Viajes en revisión previa
`/dashboard/supervisor/viaje-previo/:id`	Revisión previa de un viaje
Aprobador (rol 5)	
`/dashboard/aprobador/revisiones`	Viajes por aprobar, aprobados y rechazados
`/dashboard/aprobador/revision-alcohol`	Rendiciones con alcohol
`/dashboard/aprobador/viaje-previo/:id`	Aprobación de un viaje
Revisor (rol 4)	

Ruta	Página
`/dashboard/revisor`	Rendiciones para revisión final
`/dashboard/revisor/revision/:id`	Revisión final
`/dashboard/revisor/solicitudes-plazo`	Solicitudes de ampliación de plazo
`/dashboard/revisor/reemplazos`	Solicitudes de reemplazo
Tesorero (por cargo)	
`/dashboard/tesorero/revisiones`	Viajes por asignar fondos, aprobados y rechazados
`/dashboard/tesorero/viaje/:id`	Asignación y aprobación de fondos
Administrador (rol 1)	
`/dashboard/administrador`	Resumen general con indicadores, gráficos y filtro de periodo
`/dashboard/administrador/usuarios`	Gestión de usuarios
`/dashboard/administrador/cargos`	Gestión de cargos
`/dashboard/administrador/secciones`	Gestión de secciones
`/dashboard/administrador/historial-accesos`	Ingresos, salidas y cambios de contraseña
`/dashboard/administrador/recordatorios`	Días y horas del resumen de pendientes

Todos los perfiles tienen además sus rutas `/perfil` y `/configuracion`.

6. Gestión de sesión

6.1 Token y renovación

El token de acceso vive solo en memoria (`services/shared/tokenStore.js`), nunca en `localStorage`. Al abrir la aplicación sin token, `App.jsx` intenta un refresco silencioso con la cookie `httpOnly` antes de mostrar las rutas protegidas.

`apiClient` agrega el token a cada petición y, ante una respuesta `Token expirado`, pide uno nuevo a `/auth/refresh` y repite la petición. Si varias peticiones vencen a la vez, solo una renueva y las demás esperan en una cola:

```
if (isTokenExpired && !originalRequest._retried) {
  if (isRefreshing) {
    return new Promise((resolve, reject) => {
      waitingQueue.push({resolve, reject});
    }).then((token) => {
      originalRequest.headers.Authorization = `Bearer ${token}`;
      return apiClient(originalRequest);
    });
  }
  originalRequest._retried = true;
  isRefreshing = true;
}
```

6.2 Sesión expirada

Si la renovación falla (cookie vencida, sesión invalidada por cambio de rol, suspensión o cambio de contraseña en otro equipo), `apiClient` borra el token y emite el evento `session-expired`. `useMenu` lo escucha y muestra la ventana "Sesión Expirada", presente en todas las pantallas salvo la de ingreso; su botón cierra la sesión y vuelve al ingreso. `useMenu` además compara el rol cada 30 segundos.

6.3 Cambio de contraseña obligatorio

Cuando el servidor responde 403 con `CAMBIO_CONTRASENIA_REQUERIDO`, `apiClient` emite `password-change-required` y `PasswordChangeGate` muestra la ventana según el motivo del token:

Motivo	Texto de la ventana	Campos
`TEMPORAL`	Contraseña temporal: crea una propia	Nueva y confirmación
`RECUPERACION`	Ingresaste con un código de verificación	Nueva y confirmación
`VENCIDA`	Actualiza tu contraseña (90 días)	Actual, nueva y confirmación; ofrece cerrar sesión

Mientras está activa, los paneles ignoran los errores causados por el bloqueo para no mostrar avisos falsos.

7. Gestión de estado

No se usa una biblioteca de estado global: el estado es local a cada pantalla y vive en su hook de negocio.

- **Servidor**: se obtiene al montar y se refresca tras cada escritura.
- **Formularios**: el hook guarda los valores y los errores por campo.
- **Sondeo**: cada 30 segundos se actualizan las bandejas de revisión, las solicitudes de plazo y reemplazo, el historial de viajes del empleado y los resúmenes. Mientras el empleado tiene una solicitud

esperando respuesta, también se consulta cada 30 segundos. Solo la carga inicial muestra el indicador esquelético; si una actualización falla se conservan los datos y se muestra un aviso.

- **Concurrencia**: si otra persona ya procesó un viaje o una solicitud, el servidor responde 409 y la interfaz lo explica y actualiza la lista.

8. Capa de servicios

Servicio	Endpoints que encapsula
`user/authService`	Ingreso, cierre de sesión, códigos de recuperación
`user/userService`	Perfil, foto, contraseña, usuarios, empleados
`trip/tripService`	Panel, historial, detalle, creación, edición, envío, finalización, planilla PDF y planillas de varios viajes para Excel
`expense/expenseService`	Gastos, detalle y recibos
`expense/invoiceService`	Extracción, registro y edición de facturas
`approval/reviewService`	Revisión del supervisor (viajes y rendiciones)
`approval/approverService`	Aprobación de viajes
`approval/approverAlcoholReviewService`	Revisión por alcohol
`approval/treasurerService`	Asignación y aprobación de fondos
`approval/reviewerService`	Revisión final
`approval/deadlineAuthorizationService`	Solicitudes de ampliación de plazo
`approval/substitutionService`	Solicitudes de reemplazo
`admin/adminService`	Resumen general, usuarios, cargos, secciones, historial de accesos y recordatorios

9. Hooks de negocio

Dominio	Hooks
trip	`useEmployeeDashboard`, `useCreateTrip`, `useEditTrip`, `useTripDetail`, `useTripHistory`, `useStatementPdfDownload`
expense	`useRegisterExpense`, `useEditExpense`, `useUploadInvoice`, `useEditInvoice`, `useInvoiceForm`, `useInvoiceDetailPanel`, `useExpenseDetail`
approval	Uno por bandeja y por detalle de cada revisor (`useSupervisorPendingTrips`, `useApproverReviews`, `useTreasurerReviews`, `useReviewerReviews`, ...), más `useDeadlineAuthorization`, `useSubstitutionRequest` y sus pares del revisor; `useStatementsExport` para exportar planillas en grupo

Dominio	Hooks
admin	`useAdminDashboard`, `useUserManagement`, `usePositionManagement`, `useSectionManagement`, `useAuditLog`, `useReminderSettings`, `useAdminExcelExport`
user	`useLogin`, `useProfile`, `useCurrentUser`, `useIsTreasurer`, `usePasswordExpiredCheck`
shared	`useMenu`, `useMenuNavigation`, `useClientPagination`, `useSimpleSelector`, `useUserRole`

Ejemplo de hook con validación previa y ventana de aviso, tomado de la finalización de un viaje:

```
const handleRequestSubmitReview = () => {
  if (expenses.length === 0) {
    showError('El viaje debe tener por lo menos un gasto asociado para ser finalizado');
    return;
  }
  const {missingDays, missingHotel} = missingJustifications();
  if (missingDays.length > 0 || missingHotel) {
    showError('Debes justificar cada día que excede la cuota diaria, y el exceso en hoteles si
     corresponde');
    return;
  }
  setShowSubmitReviewModal(true);
};
```

10. Componentes

10.1 Componentes transversales

Componente	Uso
`ConfirmDialog`	Ventana emergente de confirmación o aviso; se dibuja en un portal sobre toda la pantalla
`Button`, `ModalActions`	Botones primario y secundario
`ModalIconHeader`	Ícono en círculo blanco de las ventanas
`InputField`, `PasswordField`, `FormField`, `LimitedTextarea`	Campos de formulario
`EmployeeDropdown`, `SectionDropdown`, `InlineDropdown`	Selectores con búsqueda
`ListCount`, `LoadMoreButton`	“Mostrando X de Y” y paginación

Componente	Uso
`SkeletonCard`, `SkeletonList`	Indicadores de carga
`EmptyState`, `InlineAlert`, `SuccessModal`	Estados vacíos y avisos
`PageHeader`, `AppHeader`, `AppLogo`	Encabezados
`CommentCard`	Observación de un revisor
`ExportButton`	Botón “Exportar a Excel” con estado de carga

10.2 Componentes por dominio

Dominio	Organismos principales
trip	`TripActiveExpenseView`, `TripDraftView`, `TripFinalApprovedView`, `TripBudgetSummary`, `DailyBreakdownCard`, `TripHistoryFilter`, `SubstitutionRequestModal`
expense	`InvoiceForm`, `InvoiceDropZone`, `InvoiceDetailPanel`, `ExpenseFormCard`, `ExpenseWithholdingsCard`, `ReceiptSentModal`
approval	`ReviewFilters`, `PendingTripItem`, `ExpenseInvoicedTable`, `ExpenseUninvoicedTable`, `ExpenseInternationalTable`, `SelectableObservationsList`, `TreasurerAmountsCard`
admin	`UserFormModal`, `PositionFormModal`, `SectionFormModal`, `TripPhaseSection`
user	`PasswordChangeGate`, `PasswordExpiredModal`, `SessionExpiredModal`, `VerificationCodeModal`, `TermsModal`

11. Sistema de diseño

11.1 Paleta de colores

La paleta se centraliza en `constants/index.js`; ningún componente define colores sueltos.

Color	Token	Valor	Uso
	`primary`	`#870002`	Acciones principales y ventanas emergentes
	`secondary`	`#D20F12`	Botón de confirmación, errores
	`title`	`#500203`	Títulos
	`text`	`#2E2827`	Texto
	`labels`	`#475569`	Etiquetas y texto secundario
	`background`	`#FFFFFF`	Fondo general

Color	Token	Valor	Uso
	backgroundSecondary	#000000	Íconos sobre círculo blanco
	backgroundHeader	#F3F6FF	Tarjetas y encabezados de tabla
	dataFields	#DEE2F0	Bordes y separadores
	fields	#C2C6D4	Controles deshabilitados
	error	#FDE9E9	Fondo de mensajes de error

Tipografías: **Inter** para el contenido y **Nunito** para los botones.

11.2 Colores de estado

Cada estado de viaje tiene un único nombre (`constants/tripStatusLabels.js`) y un único par de colores (`constants/tripStatusColors.js`) en todas las pantallas. Los avisos de confirmación o rechazo usan el color del estado en que queda el viaje.

Estado	Etiqueta	Significado
BORRADOR	Borrador	Aún no enviado
EN_REVISION_VIAJE	Revisión Previa	Con el supervisor
APROBADO_VIAJE	Esperando Aprobador	Con el aprobador
EN_REVISION_TESORERO	Esperando Fondos	Con tesorería
EN_CURSO	En Curso	Registro de gastos
EN_REVISION	En Revisión	Rendición con el supervisor
EN_REVISION_APROBADOR	Revisión por Alcohol	Con el aprobador
APROBADO_SUPERVISOR	Apr. Preliminar	Con el revisor final
APROBADO_FINAL	Aprobado	Cerrado
RECHAZADO	Rechazado	Debe corregirse

En las pantallas del empleado, “En Curso” usa el rojo institucional porque marca su viaje activo.

11.3 Ventanas emergentes

Todas las confirmaciones y avisos importantes usan `ConfirmDialog`: tarjeta en rojo institucional, ícono negro en un círculo blanco y botones primario y secundario. Se dibuja con `createPortal` sobre el `body`, lo que evita parpadeos cuando se abre desde una tarjeta animada.

```
<ConfirmDialog isOpen={!!result} compact
  icon={result?.success ? CheckCircle : AlertCircle} iconColor={COLORS.primary}
  title={result?.success ? 'Listo' : 'No se pudo completar'}
  message={result?.message} confirmText="Entendido" hideCancel onConfirm={closeResult} />
```

Se muestran en ventana, entre otros: los errores al finalizar un viaje, la devolución de una revisión, el resultado del envío de recibos, las respuestas a solicitudes y los avisos de solicitudes cerradas automáticamente.

12. Funcionalidades destacadas

12.1 Planilla de rendición

`useTripExcelExport` construye la planilla oficial en Excel: logotipo, simbología, distinción entre gastos nacionales e internacionales, cuenta Oracle por categoría, tramos de cambio, totales por moneda, justificaciones de exceso y cinco bloques de firma. El revisor y el tesorero exportan además las planillas de todos los viajes de su lista, según el filtro aplicado, en un solo archivo: una hoja *Resumen* con una fila por viaje y una hoja de planilla por viaje; el administrador puede exportar las del periodo elegido. El empleado descarga en cambio la planilla en PDF, generada por el backend, cuando el viaje está aprobado. Además del formato institucional, el Excel trae dos hojas con el formato que contabilidad sube a Oracle (`utils/oracleExport.js`): *FACTURAS*, con una fila por gasto y las mismas columnas de la cabecera de factura de Oracle (proveedor, fecha, número, importe, código de impuesto, grupo de retención, condiciones, grupo de pago, cuenta de pasivo, etc.), y *LINEAS*, con las líneas de distribución de cada gasto (artículo, impuesto y retenciones) y la fila de la factura a la que pertenecen. Los campos fijos llevan su valor por defecto (`constants/oracleLists.js`) y los que en Oracle son listas de valores (tipo de factura, divisa, código de impuesto, grupo de retención, condiciones, grupo de pago, método de pago, tipo de línea) se eligen con listas desplegables; se usan solo tres colores (hoja *LEYENDA ORACLE*): blanco, datos listos que pone el sistema o valores fijos; celeste y en blanco, lo llena Oracle al subir la factura (número de justificante y de proveedor, dirección, fecha contable, importes calculados, tipo de cambio, estado) y no se toca; amarillo, falta un dato que el sistema no tiene (CUF, número de recibo aún no emitido, códigos de Oracle de la sección, número de rendición), con la nota “Completar antes de subir a Oracle”. La descripción lleva la sigla del comprobante (F, R, C o A), el mes y el número de rendición del mes, que la base asigna al quedar la rendición aprobada del todo (si aún no lo tiene, sale `___` en amarillo). Las facturas van con condiciones “0D” y grupo de pago “M-CENTRO”; los recibos y los gastos sin factura, con “Contado” y “TRANS-EMPLEADOS”. Una factura con alcohol pierde el IVA y se registra como recibo. El proveedor es el nombre completo del empleado dueño de la rendición, como “APELLIDOS, NOMBRES (BOL)”. Todas las celdas llevan borde, los encabezados van centrados y cada columna se ajusta al ancho de su contenido. En LINEAS, las líneas de impuesto y retención van en celeste y cursiva: Oracle las crea al validar y quedan solo como referencia de los montos. Ambos formatos calculan las retenciones con base incrementada desde lo pagado (`utils/withholdings.js` y su equivalente del backend), igual que la tabla de revisión y el detalle del gasto. Los gastos se listan ordenados por fecha. En las facturas, el IVA va en su columna y el importe costo/gasto es el total menos el IVA; en gastos sin factura, el costo es lo pagado más las retenciones; en recibos y gastos con alcohol, el costo es el total.

12.2 Categorías de gasto

La categoría se elige entre el plan de cuentas de la empresa (112 cuentas de la planilla de Castro más “OTROS”) con `CategorySelector`, que usa el selector con buscador `SearchableSelector` de `components/ui`: se puede buscar por número de cuenta, nombre o nota de uso. Las cuentas con nota de la planilla de Castro la muestran en la lista (`DropdownListItem` con `description`) y debajo de la categoría elegida, para no confundir, por ejemplo, Taxis (viaje) con Taxis gestión oficina. La planilla en Excel ofrece la misma lista en la columna CUENTA/ORACLE (`constants/oracleAccounts.js`).

12.3 Motivo del viaje

El motivo admite hasta 500 caracteres (`constants/tripLimits.js`). Se escribe en `AutoResizeTextarea`, dentro de `InputField` con `alignTop`: empieza en una línea, como los demás campos, crece al escribir hasta seis líneas y muestra un contador. En las listas se recorta a dos o tres líneas.

12.4 Monto del viaje

Al crear o editar un viaje, `useCreateTrip` y `useEditTrip` calculan el *Monto Total Estimado* con las tarifas del cargo del empleado. En un viaje nacional es *días × tarifa en Bs*. En uno internacional, el día de salida y el de regreso se calculan con la tarifa en Bs y los días intermedios con la tarifa en USD, y la pantalla lo explica en una nota. Tesorería revisa el monto y lo ajusta al asignar los fondos.

12.5 Control de gasto diario

El *Control Diario* (`DailyBreakdownCard`) muestra una tarjeta por día con una barra por moneda y su estado: dentro del límite, cerca del límite, excedido o justificado. En un viaje nacional se controla la tarifa en bolivianos; en uno internacional, cada día con las dos tarifas por separado, y se puede gastar en cualquiera de las dos monedas cualquier día. Lo ven el empleado y los revisores. Cuando un día o el total de hoteles excede la tarifa, `useTripDetail` exige una justificación por cada exceso antes de permitir el envío a revisión.

12.6 Tipo de cambio

En los tramos de otra moneda de un gasto internacional (`ExpenseFormCard`), el empleado elige la moneda de la lista compartida (`constants/currencies.js`), escribe el monto y el tipo de cambio que le aplicaron (“1 USD = cuántos X”) y ve al momento el equivalente en dólares.

12.7 Retenciones de gastos sin factura

Bajo las retenciones de un gasto sin factura, la tarjeta `WithholdingSummary` (la misma en el registro y la edición) muestra el monto pagado, el RC-IVA o IUE, el IT y el importe costo, calculados en `utils/withholdings.js` con base incrementada: el importe costo es lo pagado $\div$ 0.84 en servicios ($\div$ 0.92 en compras) y de él salen las retenciones, de modo que pagado + retenciones = importe costo (por ejemplo, 40.00 + 6.19 + 1.43 = 47.62). `WithholdingFormulaNote` lo explica en pantalla.

12.8 Digitalización de comprobantes

La carga de facturas presenta cada comprobante en tres columnas (imagen, formulario y detalle de productos) para corregir los datos extraídos antes de guardar. Cada factura tiene su propio estado: en extracción, con datos, con error o guardada. Al registrar, la fecha de emisión siempre se puede corregir, por si la IA la leyó mal; al modificar una factura guardada (`EditInvoicePage`, que usa `InvoiceForm` con `lockDate`) queda bloqueada. El empleado escribe el total que figura en el papel (“Total de la factura”): con Factura (F) el IVA es el 13 % del total y se resta de él (`utils/invoiceAmounts.js`: IVA = total $\times$ 0.13 y subtotal = total $-$ IVA), y el formulario muestra ambos; con Recibo (R) no hay IVA y el total

es lo pagado, también en la planilla en Excel. Al cambiar entre F y R se conserva el total pagado. El campo “Número de Factura / Código” admite símbolos (@, \protect \T1\textdollar , #) para los códigos de recibos y boletos. En el detalle de productos (InvoiceDetailPanel) la cantidad admite hasta 3 decimales, por ejemplo 53.345 litros de combustible o 6 unidades, y se muestra sin ceros de más (formatQuantity). La factura guarda también el CUF (código de autorización): la IA lo lee del comprobante y el empleado puede corregirlo en el campo “CUF / Código de Autorización” (solo letras y números); el recibo no lo pide. Los gastos sin factura tienen un campo opcional “N° de Recibo” para el número impreso en el recibo en papel, que el Excel para Oracle usa como número de factura (si está vacío, usa el del recibo que emite el sistema). Además, admiten, junto al comprobante, hasta 2 respaldos opcionales, por ejemplo el comprobante del cambio de moneda (BackupImagesUpload), que se ven en el detalle del gasto y se pueden quitar o cambiar al editarlo. Las imágenes se comprimen en el navegador antes de enviarse.

12.9 Revisión y observaciones

Cada bandeja de revisión tiene las pestañas Pendientes, Aprobados y Rechazados, con filtros por fecha, empleado y sección, y muestra lo revisado más reciente primero, de 12 en 12. Para rechazar se seleccionan observaciones propias de la ronda actual; las de otros revisores se ven, pero no se pueden editar, borrar ni usar para rechazar. El supervisor y el aprobador pueden devolver una revisión tomada por error.

12.10 Plazos y reemplazos

useDeadlineAuthorization determina si venció el plazo de carga (cuatro días de tolerancia más las ampliaciones vigentes) y ofrece pedir una ampliación. El plazo es del viaje, por lo que el reemplazo ve y puede pedir la misma ampliación. useSubstitutionRequest gestiona el pedido de reemplazo; el viaje aparece en el panel del reemplazo como “Rendición de [nombre]”. Las solicitudes cerradas por el sistema se muestran como “Cerrada” y no como rechazadas.

12.11 Administración

Resumen general con indicadores por fase y gráficos, con filtro de periodo (este mes, que es el valor inicial, este año, todo o un rango, por fecha de inicio del viaje); el gráfico por sección cuenta solo los viajes con fondos entregados por tesorería; gestión de usuarios con buscador en cargo, sección y jefe directo (SearchableSelector); sugerencias de nombres existentes al crear cargos y secciones; en cada sección, sus códigos de Oracle (dependencia y sección, 3 dígitos), que forman la cuenta del gasto en la hoja LINEAS del Excel para Oracle (si faltan, se usan 900 y 000 y la celda queda en amarillo); historial de accesos con filtro de periodo (por defecto el mes actual, en días de Bolivia; ambos usan los mismos botones de periodo, PeriodPresets), filtros por evento y usuario, paginación de 20 en 20 y exportación a Excel; la exportación a Excel del Resumen General, Usuarios, Cargos y Secciones (useAdminExcelExport), con el mismo estilo; y la pantalla de *Recordatorios*, donde el administrador activa o desactiva el resumen de pendientes, elige los días de la semana y hasta cuatro horas de envío (hora de Bolivia), ve una vista previa de a quién le llegaría y puede enviarlo en el momento. Los cambios se aplican al guardar, sin reiniciar el servidor.

12.12 Casquito

Mascota que saluda una vez por pantalla y sesión con un mensaje según la hora y los pendientes. Se puede desactivar en Ajustes de Cuenta y respeta la preferencia de reducir movimiento.

13. Documentación de componentes

Los componentes de `components/ui`, todos los átomos de `features`, las moléculas reutilizables y los organismos de la carga de facturas (`InvoiceForm`, `InvoiceDetailPanel`) están documentados en Storybook, con cada historia junto a su componente (137 historias en total). Cada historia describe el propósito del componente y sus variantes; los que tienen estado interno (selectores, diálogos) incluyen un contenedor interactivo para probarlos.

```
npm run storybook          # http://localhost:6006
npm run build-storybook    # sitio estático de la documentación
```

14. Dependencias y seguridad

`npm audit` no reporta vulnerabilidades. `package.json` fuerza (`overrides`) la versión 11 de `uuid` para ExcelJS, que solo usa su función `v4`; el navegador usa la versión empaquetada de ExcelJS, por lo que el Excel sale igual.

15. Pruebas automáticas

Una prueba automática es código que usa el sistema como lo haría una persona y verifica que el resultado sea el correcto. Se ejecutan con un comando y, si un cambio rompe algo, fallan antes de que llegue a producción.

Tipo	Qué prueba	Comando
Unitarias	Hooks, componentes y utilidades: estados del control diario, cantidades con decimales del detalle de factura, periodos, exportación a Excel y hojas para Oracle. Cada archivo de prueba está junto a lo que prueba (`*.test.js`)	`npm test`
De extremo a extremo	El sistema completo en un navegador real, en escritorio y celular: ingreso de cada rol, monto calculado del viaje, control diario con justificación, tipo de cambio de los tramos, respaldos de gastos sin factura y exportaciones del administrador (`e2e/`)	`npm run test:e2e`

Ejemplo de prueba de extremo a extremo: el sistema calcula el monto de un viaje de tres días con la tarifa del cargo y lo guarda como borrador.

```
test('el sistema calcula el monto del viaje con la tarifa del cargo', async ({page}) => {
  await loginInBrowser(page, 'juan.empleado');
  await page.goto('/dashboard/empleado/crear-viaje');
  await page.getByPlaceholder('Inspección técnica y de producción...').fill('Prueba de monto');
  await page.getByPlaceholder(/Ciudad de origen/).fill('La Paz');
  await page.getByPlaceholder('¿A dónde se dirige?').fill('Santa Cruz');
  const dates = page.locator('input[type=date]');
  await dates.nth(0).fill(boliviaDate(5));
  await dates.nth(1).fill(boliviaDate(7));
  await expect(page.getByText(/900\.00/).first()).toBeVisible();
  await page.getByRole('button', {name: 'Guardar Borrador'}).click();
  await expect(page.getByText('Se registró el viaje correctamente')).toBeVisible();
});
```

Las pruebas de extremo a extremo necesitan el backend y el frontend levantados sobre la base de prueba; `E2E_BASE_URL` y `E2E_API_URL` indican dónde están. En el estado actual pasan 40 pruebas unitarias y 20 de extremo a extremo.

15.1 Integración continua

El archivo `.github/workflows/pruebas.yml` ejecuta las pruebas en GitHub Actions en cada envío y en cada pull request: compila la aplicación y Storybook, corre las pruebas unitarias y, para las de extremo a extremo, descarga el backend de la misma rama, crea una base PostgreSQL temporal y levanta todo. Si algo falla, el reporte con capturas queda en la pestaña *Actions*.

16. Compilación y despliegue

1. Definir `VITE_API_URL` con la dirección del backend de la etapa.
2. Verificar que `src/assets/foto-login.jpg` esté en el repositorio.
3. En Vercel: proyecto con Vite, comando `npm run build` y carpeta `dist`. Cada cambio de variable requiere volver a desplegar.
4. En un servidor propio: `npm run build` y servir `dist/` con Nginx, redirigiendo todas las rutas a `index.html`.
5. Agregar el dominio del cliente en `CORS_ORIGINS` del backend si no es el de Vercel.

17. Convenciones

- El código se escribe en inglés y los textos visibles en español.
- Los componentes definen sus estilos en un objeto `styles` con clases de Tailwind y toman los colores de `COLORS`.
- En el frontend no se agregan comentarios por función; el nombre y el contenido deben bastar. Solo se admite un comentario breve, de una línea, donde aclara algo no evidente.

- Se prefieren bloques `if`/`else` explícitos; el operador ternario solo se usa en valores de estilo o del JSX.
- Punto y coma al final de cada sentencia y literales de objeto sin espacios internos (`{dato: valor}`).
- Los commits siguen Conventional Commits: `feat`, `fix`, `style`, `refactor`, `docs`, `test` y `chore`.